AI-Assisted Development

Rules for long-lived and new projects

An adaptable template — binding policy and best practices

A framework for managing the risks of AI-assisted development, to adopt and adapt
Version v0.1.3 (4f6b057) — 2026-11-07

Contents

1 How to read this document **3**

2 Premise: why the rules differ with the age of the project **3**
2.1 A1. Rules that apply to all projects . . . 5
2.2 A2. Policy for long-lived / established projects . . . 6
2.3 A3. Policy for new projects . . . 6
2.4 A4. Responsibility matrix by blast radius and project type . . . 6
2.5 A5. Enforcement and review of the boundaries . . . 7
2.6 A6. Escalation on a breach . . . 8
2.7 B1. For long-lived / established projects . . . 9
2.8 B2. For new projects . . . 9
2.9 B3. Shared practices . . . 9
2.10 B4. Modes of working with AI on data under NDA . . . 9
2.11 B5. Layered architecture as a precondition for a wide AI zone . . . 10
2.12 B6. Protecting know-how and the outbound data flow . . . 11

3 Onboarding checklists (step by step) **11**
3.1 Checklist 1 — Introducing AI into an established project . . . 11
3.2 Checklist 2 — Starting a new project with AI . . . 12
3.3 Checklist 3 — Before every merge of an AI-assisted MR . . . 12

4 Categorising a project (how to classify it) **12**
4.1 Established project . . . 12
4.2 Borderline project (new code in an established context) . . . 13
4.3 New project (greenfield) . . . 13

5 Appendix: ADR template **13**

6 Quick rules to close **13**

1. How to read this document

This is a template, not a finished policy

The document is a framework to adopt and adapt, not the policy of a particular organisation. Places such as “project leadership decides” or the escalation authority are deliberately generic — fill them in according to your own structure of roles. Take what fits; adapt or drop the rest. It is distributed under the CC-BY 4.0 licence (free use including commercial, with attribution).

What the rules enforce and what they do not

The rules in this document do not enforce judgement; they only mark the places where judgement is required and who is responsible for it. The mechanics (the MR gate, `CODEOWNERS`, CI checks — see A5) are enforceable, but not sufficient on their own. The core is rules A1.1 and A1.2, that is, understanding the code and taking ownership of it — and these cannot be enforced, only required in review. This is intentional, not a shortcoming: the rules are written with the intent that responsibility stays with people and does not shift to the tool or to the process. For adopting the framework it follows that an organisation which adopts the mechanics without taking on the responsibility has gained nothing — it gets green checks with no change in how decisions are made. The framework is meant to count as deployed only when there is a specific, named person willing to reject an MR by citing A1.1, and that person has cover to do so. If there is no such person, the framework is not deployed yet, whatever the repository contains.

The document has two clearly separated parts. Do not confuse them — they differ in how binding they are.

PART A — Policy (binding)

Rules that MUST be followed. A breach is grounds for rejecting a merge request, and in risk-bearing areas it can mean regulatory exposure (NIS2/GDPR). **Wording:** “must”, “must not”, “is prohibited”.

PART B — Best practices (recommendations)

Practices that raise quality and lower risk, but remain subject to engineering judgement. Deviation is acceptable when it is deliberate and defensible. **Wording:** “we recommend”, “appropriate”, “consider”.

Two project domains: the rules differ depending on whether this is a **long-lived / established project** (*legacy* — it exists, it runs, it has history), or a **new project** (greenfield, where the boundaries are still being drawn).

Borderline case

An actively developed established project = ESTABLISHED. What decides is the existence of the codebase and its blast radius, not the intensity of current development. A new module inside an established system follows the rules for established projects until it is isolated behind a stable contract (interface).

2. Premise: why the rules differ with the age of the project

The difference is not arbitrary. It follows from two factors that differ fundamentally between an established and a new project:

- **Mental model.** In an established project there is documented and undocumented knowledge that AI does not have and cannot reconstruct from the code alone. Just as AI does not know

the undocumented context of the code, it does not know the undocumented context above it either — the project, the infrastructure, the organisation and everything beyond, as far as the decisions reach. Code is only the most visible layer; below it and above it lie the reasons you cannot read out of the artefacts themselves — why something is built the way it is, what once broke, which dependencies must not be touched. In a new project the model is only being formed — and AI can quietly replace it before it ever comes into being.

- **Blast radius.** It denotes the extent of the damage caused by a fault in a given part of the code. In an established project it is already fixed and often large (integrations, data, dependencies). In a new project it can still be designed so that the risk-bearing parts are small and isolated.

The size of the damage is not the whole risk picture, however. Just as important is how quickly we *detect* a fault and how easily we *undo* it. We therefore measure the risk of a given part of the code along three axes:

- **Blast radius — how large the damage is.** How much fails when it goes wrong (from a local bug to a systemic security incident).
- **Verifiability — how easily we can tell that it is wrong (or right).** A fault in domain logic fails a test straight away; a fault in encryption does not show up in a test, but as a leak months later. Poor verifiability = the fault waits quietly for a long time.
- **Reversibility — how easily we undo it when we get it wrong.** A feature flag or a reversible migration lowers the risk even with a large blast radius; deleted data without a backup is irreversible even with a small one.

> The axes combine — the worst case is a large blast radius *and* poor verifiability *and* irreversibility (the fault is large, we do not see it for a long time, and it cannot be taken back). Conversely, a large blast radius that is easy to verify and easy to reverse (feature flag, quick rollback) is manageable. This is why the degree of direct developer control (A1.3) is driven not only by the size of the damage, but by all three axes together.

> Consequence: in an established project the rules protect EXISTING understanding from erosion. In new projects they make sure understanding comes into being at all, and that the risk boundaries are drawn correctly from the start.

Factor	Long-lived / established	New project
Mental model	Exists, is valuable, AI does not know it	Still forming, AI can crowd it out
Blast radius	Fixed, often large	Can be designed small and isolated
Main AI risk	Silent regression, breaking guaranteed assumptions	A developer without a model of their own code
Role of the rules	Protect understanding from erosion	Ensure understanding comes into being
Width of the AI-led zone	Narrower — only isolatable tasks	Wider — once the skeleton and ADRs exist

Blast radius is a design variable, not fate

> A positive consequence follows from the table: blast radius is not fixed — it is largely a *result of the architecture*. With a well-layered design, where the security and data layers are isolated behind stable contracts, the risk-bearing core shrinks to a small, well-guarded

> surface. Most of the code then lies in the low-blast-radius zone, where AI can work almost without restriction. Architecture is therefore not only a defence against AI, but also *a way to let AI safely go further* — developed further in B5.

PART A — Policy (binding)

2.1 A1. Rules that apply to all projects

They apply regardless of the age of the project and take precedence over everything below.

A1.1 The understanding rule. Code that the developer does not understand and cannot defend without AI MUST NOT be merged — regardless of the fact that it "works".

A1.2 Ownership. The question is not whether AI wrote the code, but whether the developer has fully taken ownership of it. That means: understanding it, being able to explain its trade-offs, backing it with tests — and in risk-bearing areas with a threat model as well — and putting it through the appropriate review. Whatever meets these conditions may be used even in risk-bearing areas; whatever does not must not be used anywhere. The exception is secrets management (A1.4), where the ban is absolute regardless of ownership.

A1.3 Risk drives control. The greater the risk — the size of the damage, the poorer the verifiability and the poorer the reversibility (see Premise) — the higher the mandatory degree of direct developer control. Classification follows the matrix in A4.

A1.4 Secrets and keys. AI MUST NOT generate the handling of credentials, keys and tokens (secrets management). No credentials, keys or tokens in the code or the repository — never, in any project. This is not about understanding but about the fact that working with real secrets through AI is itself a leak of them; hence the exception from principle A1.2.

A1.5 Marking the origin. AI-generated code must be marked in the commit or the merge request (e.g. `[ai-generated]`) for auditability.

A1.6 The security layer — the highest bar for ownership. For authentication, authorisation, encryption, persistence, OAuth tokens and audit logging, principle A1.2 applies in its strictest form: no matter who designed the code, the developer must take full ownership of it — back it with a threat model and tests, and defend every decision without AI. Without that ownership the code must not enter these areas, even if it "works".

A1.7 A data processing agreement (DPA) before any data is sent. No personal data is sent to an external AI interface without a valid Data Processing Agreement (DPA) — and, for services outside the EU, without Standard Contractual Clauses (SCC). Without a documented legal basis for the transfer the tool is not used; assessing that basis belongs to the role responsible for personal data protection.

A1.8 Data under NDA and untrusted AI tools. An AI tool that sends code or data to a third party for processing is, for the purposes of this policy, considered a *transfer of data to a third party* — regardless of the fact that the tool offers a "do not store" or "do not train on my data" option. A provider declaration is not a verifiable guarantee. Therefore: for data covered by an NDA, project leadership must *explicitly decide on the mode of working with AI* (see B4) BEFORE the tool is used. The default mode in the absence of a decision is the strictest one (sending prohibited).

A1.9 The configuration of the AI tool is part of the code. Files that govern the behaviour of an AI tool (rules, system instructions, development environment configuration) are subject to the same review as the code they influence, and belong in the register of risk-bearing paths under A5.2. AI may propose them, but must not introduce them — changing these files

without ownership under A1.2 is a breach of this policy. What is not immediately visible in the diff is checked as well (invisible characters, a change of encoding). Reason: whoever changes the rules changes all the rules at once — including those meant to catch that very change.

2.2 A2. Policy for long-lived / established projects

This concerns systems that are in production, have history and a fixed blast radius.

A2.1 No unowned writes into an established codebase. AI must not write into an established codebase autonomously — its output enters as a proposal, which the developer takes ownership of under A1.2 (understands it, backs it, defends it), not as a direct commit. Where the sensitivity of the project requires it, results are handed over solely as textual instructions for the development environment, not as finished files.

A2.2 No blanket AI refactoring. Renaming integration properties, changing contracts and bulk refactors are not done with an AI tool across the whole codebase. This pattern repeatedly leads to high remediation costs (on the order of person-days).

A2.3 A contract change = an ADR. Any change to a public contract (API, integration interface, schema) requires an Architecture Decision Record (ADR) BEFORE implementation, regardless of who or what wrote the code.

A2.4 The reviewer verifies the guaranteed assumptions. In an established project the reviewer asks for an explanation of why the change does not break the assumptions the rest of the system (and the integrations downstream) relies on — not merely that the tests passed.

A2.5 AI assists, the developer takes ownership. The default in an established codebase is that AI advises and the developer takes ownership of the output under A1.2. The AI-led mode (wider latitude) is an exception, permitted only for isolatable, easily verifiable tasks (see A4).

2.3 A3. Policy for new projects

Greenfield is an opportunity to draw the boundaries correctly. The rules aim to let AI speed up the start without replacing the formation of the mental model and the architecture.

A3.1 The skeleton and the boundaries first. The architectural skeleton and the blast-radius boundaries (where security, data and integrations lie) are defined by the developer BEFORE AI starts generating the filling.

A3.2 ADRs from commit no. 1. Key decisions (persistence, authentication, encryption, module boundaries) have ADRs from the beginning. A new project has no “it is historically that way” excuse.

A3.3 Isolation of the risk-bearing parts. The security and data layers are designed to be small, isolated and testable, so that the AI-led zone can safely be wider.

A3.4 Bus factor from the start. Even in a new project there must be at least one other person able to take over the code. AI does not count towards this factor.

A3.5 A wider AI-led zone only once the above is met. Only once the skeleton, the ADRs and the isolation exist may AI lead development of the domain and UI layers to a wider extent than in an established project.

2.4 A4. Responsibility matrix by blast radius and project type

A combined matrix: where AI leads, where the developer leads and who the mandatory reviewer is. It differs by project domain. “AI-led” = AI generates, the developer verifies; “developer-led” = AI may assist, but the developer must take full ownership of the output under A1.2 (understands

it, backs it with a threat model and tests, defends it without AI). The only exception is secrets management — there AI must not generate at all (A1.4).

Area	Radius	Vf / Rv	Established	New project	Reviewer
Domain / business logic	Low	+ / +	AI proposes, developer verifies	AI-led	Developer
UI components, forms	Low	+ / +	AI proposes	AI-led	Developer
Tests, documentation	Low	+ / +	AI-led	AI-led	Developer
API endpoints, input validation	Medium	+ / ~	Mixed, contract by hand	Mixed	Senior developer
Integration contracts / properties	Med.–high	~ / −	Developer-led + ADR	Developer-led + ADR	Senior developer
Authentication, authorisation	High	− / ~	Developer-led	Developer-led	Senior + Security
Encryption, keys	Critical	− / −	Developer-led	Developer-led	Senior + Security
Persistence, DB migrations	High	~ / −	Developer-led	Developer-led	Senior developer
Secrets management	Critical	− / −	AI must not generate	AI must not generate	Senior + Ops
CI/CD, infrastructure as code	High	~ / +	AI for routine scaffolding only	AI for routine scaffolding only	Senior + Ops

Vf / Rv = verifiability / reversibility: + good, ~ partial, − poor. How to read it: encryption has − / − — a fault there is hard to detect and hard to undo, so the developer leads regardless of everything else. CI/IaC has a large radius, but ~ / + (reversible by rollback), so AI may at least do the routine scaffolding.

Reading the matrix: the difference between established and new is most pronounced in the domain and UI layers: in a new project AI may lead, in an established one it may only propose. The security and data layers are developer-led in both cases — the risk does not depend on the age of the project.

2.5 A5. Enforcement and review of the boundaries

Rules that cannot be enforced will be bypassed by a lazy process. Enforcement rests on two layers: the process layer (mandatory, works even without strong infrastructure) and the technical layer (recommended reinforcement, introduced as means allow). The rules are *impersonal and symmetrical* — they apply equally to a junior, a senior and an architect. The gate does not check people, it checks artefacts at the boundary.

A5.1 The process layer (mandatory). Every change to the codebase goes through a mandatory merge request (MR); pushing directly to the main branch is prohibited. The reviewer MUST reject an MR that does not meet Part A — in particular one without an explanation of the design decision, without an ADR for a contract change, or with code the author does not understand. “It works” is not sufficient justification for a merge.

A5.2 A register of risk-bearing paths. The paths (directories, modules) falling into the developer-led zone (security, data, secrets, persistence, CI/IaC) are listed explicitly in the repository. A change in these paths requires the corresponding reviewer under matrix A4 — regardless of the size of the change.

A5.3 The technical layer (recommended, as means allow). Where the infrastructure permits, the process rules are reinforced automatically: a `CODEOWNERS` file binds risk-bearing paths to mandatory reviewers; a CI gate (a check job) verifies the presence of an ADR for a contract change and the marking of origin (`[ai-generated]`). The absence of this layer does NOT void the process obligations — it merely fails to replace them with automation.

A5.4 Periodic review of the boundaries. Not only individual MRs but the boundaries themselves are reviewed (quarterly is recommended): Does the line between "AI-led" and "developer-led" still hold? Has the risk-bearing core seeped into ordinary code (shared state, security scattered around)? Has the blast radius grown in parts that used to be low-risk? The review is carried out by a role with an overview of architecture and security, independently of the authors of the changes.

A5.5 No circumvention. Deliberately bypassing the gate (e.g. splitting a risk-bearing change into small MRs, disabling a check, falsely marking the origin) is a breach of the policy in itself — regardless of whether the resulting code "works".

2.6 A6. Escalation on a breach

Escalation is graduated and aims at *remediation and structure*, not at punishing a person. The goal is to bring the artefact back into line with the rules, not to find someone to blame.

Lvl	Situation	Response	Who
1	Unintentional breach	Warning in the MR + a request for remediation (explanation, ADR, rewrite). The MR is not merged until it complies.	Reviewer
2	Repeated / not remedied	Block on the MR and the branch; the change is not merged. The reason is recorded.	Reviewer + senior
3	Deliberate bypassing of the gate	Escalation to the authority; retrospective review of the affected changes.	[escalation authority — fill in]
4	Leak of data / know-how	Immediate stop; handled as a security incident (see B6, NIS2/GDPR).	[escalation authority] + Security

- **The default is remediation, not punishment.** Levels 1–2 are an ordinary part of review; they are not a sanction but the standard pressure for quality.
- **A harder response only for intent and leaks.** Levels 3–4 concern deliberate circumvention and outbound data flow — where know-how or regulatory compliance is at stake.
- **The escalation authority.** In this draft it is designated generically as `[escalation authority]`; the specific role or person is filled in when the document is approved.

PART B — Best practices (recommendations)

Recommendations are subject to judgement. Deviation is fine as long as it is deliberate and defensible.

2.7 B1. For long-lived / established projects

- **AI as a sparring partner.** The greatest benefit of AI in an established project is in analysis: explaining unfamiliar code, finding the impact of a change, generating hypotheses while debugging — not in writing production code.
- **Characterisation tests before you touch anything.** Before modifying an uncovered area, have AI propose tests describing the current behaviour. This lowers the risk of a silent regression.
- **Incremental batches.** Small, separate changes, each with its own MR. A large AI-generated diff is hard to review in an established project and erases the mental model.
- **Domain intuition out loud.** When mentoring, have the junior explain why an AI proposal is (in)appropriate in the given context — transferring intuition is worth more than the diff itself.

2.8 B2. For new projects

- **Architecture by humans, filling with AI.** Design the skeleton and the interfaces yourself (by all means in dialogue with people), and only then speed up the implementation inside the modules with AI.
- **Expect drift (~20 %).** The implementation will diverge from the original design. Capture the divergence in an ADR or a journal so that it stays deliberate and documented.
- **A walking skeleton before breadth.** First a thin working vertical slice through all the layers, and only then expansion with AI. It verifies the blast-radius boundaries in practice.
- **Pace deliberately.** Generation speed is not the goal. In a new project it is cheap to slow down and draw the boundaries correctly — fixing them later is expensive.

2.9 B3. Shared practices

- **A lightweight ADR process.** Markdown in the repository, not a ten-page Word document. It records the why, the alternatives and the trade-offs — see the appendix.
- **A developer journal.** Short notes on the direction and the reasons. It complements ADRs with running context.
- **Name the role.** Deliberately distinguish whether you are the architect, the crisis manager, the consultant or the developer — and adapt how you use AI accordingly.
- **No friction = beware.** When the loop with AI has no friction and merely confirms everything, confirmation bias is built into the process. Deliberately insert a critical step.

2.10 B4. Modes of working with AI on data under NDA

This is an area that still needs to be resolved — the modes below are a decision scale, not a finished policy. Rule A1.8 requires that for data covered by an NDA **project leadership chooses one of the modes** before an AI tool is used. The modes are ordered from the strictest to the most permissive; the choice is a trade-off between strictness and usability.

> **The core of the problem:** an “AI in the editor” type of tool sends content to the provider’s server for processing. The “do not store data” option is a declaration, not a verifiable technical guarantee — and with data under NDA, responsibility for the transfer always lies with the sender.

#	Mode	Substance	Trade-off
0	Sending prohibited	AI is not used on data under NDA at all; only on unprotected parts.	Highest certainty, lowest benefit from AI.
1	Local / self-hosted model	The model runs on your own infrastructure, data never leaves the perimeter.	Operating and hardware costs; a weaker model than the cloud.
2	Anonymisation / abstraction of the input	Only the generalised problem goes to the AI, without protected identifiers and context.	Laborious, with a risk of residual identifiability.
3	Isolated sandbox + manual transfer	AI development happens on the side on a neutral task; the result is carried over and rewritten by hand into the protected codebase.	Double the work, but protected code is never sent.

- **Project leadership decides, not the individual developer.** Choosing the mode is a decision with legal consequences; it belongs at the level that is accountable for the NDA.
- **The default is mode 0.** If no decision has been made, sending is prohibited.
- **Mode 3 as the practically available compromise.** An isolated sandbox with manual transfer back makes it possible to use AI while protected code never leaves the perimeter — the price is double work and discipline during the rewrite.
- **Combining modes.** The modes are not mutually exclusive: you can, for example, anonymise (2) the input to a local model (1).

2.11 B5. Layered architecture as a precondition for a wide AI zone

Rule A1.3 says that the degree of control is determined by the risk. The positive inversion (hinted at in the Premise) reads: *blast radius can be reduced by design* — and thereby the space in which AI may lead can legitimately be widened. This is not an excuse for looser rules, but a recipe for *earning* looser rules through good design.

> The goal: pull the risk-bearing core (security, data, integrations) into a small, isolated and well-guarded surface. What remains outside — domain logic, UI, transformations — then lies in the low-blast-radius zone, where AI can work almost without restriction and where a fault is local and easy to verify.

What creates such an architecture:

- **Stable contracts at the boundaries.** Risk-bearing layers hidden behind a clear interface that does not change with every change of logic. AI works against the contract, not inside the sensitive implementation.
- **Isolation of the security and data layers.** Authentication, encryption, persistence and secrets management in separate, small modules that everyday development does not touch. These modules are written by the developer (see matrix A4); the rest of the system is independent of them.
- **Clean separation of layers.** Domain logic knows nothing about persistence or transport. As a result, a large part of the code is testable and safe to generate without being able to break the assumptions the risk-bearing core rests on.

- **The blast-radius boundary as a deliberate decision.** During design it is explicitly determined where the line between "AI-led" and "developer-led" runs — and this line is recorded in an ADR, not implicit.

The limits of this principle

It holds only where the isolation is *real*, not merely declared. If the risk-bearing core seeps into the rest of the system (shared state, a contract that changes with every feature, security scattered across the code), then "low blast radius" does not exist and a wide AI zone is an illusion. A wide AI zone is therefore *a reward for demonstrably good design*, not the starting state.

2.12 B6. Protecting know-how and the outbound data flow

The rules in Part A mainly guard *what enters* the codebase. Just as important is *what leaves it* — an AI tool is a two-way channel, and a company's know-how leaks out through the same pipe that help flows in through.

> Sending code or context to an external AI tool is an *outbound data flow*. It is covered by rule A1.7 (DPA), A1.8 (NDA) and the modes in B4 — but even where neither personal data nor an NDA is involved, it is know-how whose value lies in the competition not having it.

- **What is know-how, not just code.** Domain models, integration patterns, solutions to non-standard problems, architectural decisions — that is the value the company has accumulated. Sending it to a tool that learns from the data or retains it is a silent drain of that value.
- **The principle of least necessary context.** Only as much context as the task requires goes to the AI — not the whole module "just in case". The less that leaves, the smaller the exposure.
- **The sensitive core is not sent.** Parts carrying competitive advantage (key algorithms, domain specifics) are handled in a mode that does not send them out (B4 — local model, anonymisation, isolated sandbox).
- **Symmetry with the inbound side.** Just as input into the risk-bearing core must be fully owned by the developer (A1.2), the risk-bearing core must not be sent out uncontrolled. The boundary holds in both directions.

3. Onboarding checklists (step by step)

Practical steps for starting to work with AI on a project. Tick them off in order; each step must be done before the next.

3.1 Checklist 1 — Introducing AI into an established project

☐ Identify the blast-radius zones (security, data, integrations) and mark them as developer-led.

☐ Verify that no credentials or keys are in the repository; if they are, handle it as an incident before anything else.

☐ Check that a DPA/SCC exists for every external AI interface that personal data could flow to.

☐ If the data falls under an NDA, verify that project leadership has chosen a mode of working with AI under B4.

☐ Set things up so that AI output enters as a proposal or a textual instruction, not as a direct write of finished files.

☐ Define which tasks are isolatable and may be AI-led (tests, documentation, local domain logic).

☐ Make sure characterisation tests exist for the areas you are about to touch.

☐ Agree with the reviewer that contract changes require an ADR and an explanation of the impact on the guaranteed assumptions.

☐ Check the bus factor: is there a second person able to take over the affected part?

3.2 Checklist 2 — Starting a new project with AI

☐ Write down the architectural skeleton and the module boundaries before AI generates anything.

☐ Mark the risk-bearing zones on the skeleton (authentication, encryption, persistence, secrets) as developer-led.

☐ If the data falls under an NDA, verify the chosen mode of working with AI under B4.

☐ Create `docs/adr/` and write ADR-0001 for the most important decision (persistence/security).

☐ Build the walking skeleton — a thin vertical slice through all the layers — by hand or with minimal AI assistance.

☐ Set up secrets management (e.g. a Key Vault) by hand; AI does not touch it.

☐ Prepare the CI/CD scaffolding; AI may do routine boilerplate only, checked by Ops.

☐ Only now open the wider AI-led zone for the domain and UI layers.

☐ Establish the habit of a journal/ADRs to capture divergence from the design.

3.3 Checklist 3 — Before every merge of an AI-assisted MR

☐ I understand every block and can defend it without AI.

☐ The MR or commit is marked as AI-generated, if it is.

☐ The change does not reach into the developer-led zone without the corresponding reviewer.

☐ If a contract changes, an ADR exists.

☐ No secrets or keys in the diff.

☐ For data under NDA, the chosen mode of working with AI was followed.

☐ Only the necessary context went to the AI, not the sensitive core carrying know-how.

☐ The MR does not bypass the gate (no splitting of a risk-bearing change, no false marking of origin).

☐ The tests describe behaviour, not merely that the code runs.

4. Categorising a project (how to classify it)

Instead of listing specific projects (that will be the subject of a separate example document), what follows is a decision guide for which category a project belongs to.

4.1 Established project

- **Signs:** it is in production, has history, existing integrations and a fixed blast radius.
- **Mode:** fully governed by Parts A2 and B1. AI consults, the developer writes.
- **Beware:** blanket AI refactoring of integration properties without an ADR is a typical source of costly remediation.

4.2 Borderline project (new code in an established context)

- **Signs:** newly written code that is nevertheless tightly coupled to an established system (shared contracts, data, integrations).
- **Mode:** until it is isolated behind a stable contract, it follows the rules for an ESTABLISHED project.
- **Opportunity:** being new code, it has a chance to draw the boundaries and version the contracts correctly from the start.
- **ADR:** decisions about the shape and versioning of the contracts belong in an ADR from the first commit.

4.3 New project (greenfield)

- **Signs:** no existing codebase or integrations; the boundaries are still being drawn.
- **Mode:** the full regime of Parts A3 and B2.
- **Sequence:** skeleton → ADR → walking skeleton → isolation of risks → a wider AI-led zone.

5. Appendix: ADR template

An ADR (Architecture Decision Record) = a short record of an architectural decision. A Markdown file in the repository, typically `docs/adr/0001-title.md`. It records the why, not the what.

```
# ADR-0001: <decision title>

## Status
Accepted - YYYY-MM-DD

## Context
What leads us to the decision; relevant constraints (e.g. GDPR Art. 32).

## Decision
What we chose and how, specifically.

## Alternatives
- Option A - why not
- Option B - why not

## Consequences
- Dependencies this creates
- Trade-offs we accept
```

6. Quick rules to close

- **You do not know exactly what the code does** → do not use it.
- **It affects security/data** → you may use it only if you take full ownership (A1.2): threat model, tests, a defence without AI. Secrets not even then (A1.4).
- **You cannot explain it without AI** → do not merge it.
- **Established codebase** → AI proposes, the developer writes.
- **New project** → skeleton and ADRs first, only then a wider AI-led zone.
- **Data under NDA** → without a leadership decision on the mode (B4), AI is not used.
- **A well-isolated risk-bearing core** → AI may lead the rest almost without restriction (B5).

- **Everything through a mandatory MR** → “it works” is not a reason to merge (A5).
- **Bypassing the gate** → a breach in itself, and it escalates (A5/A6).
- **Know-how is an outbound flow** → the sensitive core is not sent out (B6).